only two functions that will increment this internal counter. Be warned that this is not officially documented functionality; it could change without warning in the future.

### mb_substitute_character()

The previous function swapped out invalid bytes for a question mark character, which has the (Unicode decimal) ordinal value *63* and corresponds to the `substitute_character` value obtained from `mb_get_info()`. A different Unicode ordinal can be set with `mb_substitute_char()`, which can be a pain to translate from the hexadecimal values we normally use when looking at Unicode codepoints, so remember we can just use a hexadecimal literal value:

```
$invalid2Octet = "\xC3\x28";
print mb_substitute_character() . PHP_EOL;
mb_substitute_character(0xFFFD);

$converted = mb_convert_encoding(
    "Te{$invalid2Octet}st",
    to_encoding: 'UTF-16BE',
    from_encoding: 'UTF-8',
);
print bin2hex($converted);

Output:
63
00540065fffd002800730074
```

The first call to `mb_substitute_character()` has no argument, so the function returns the current ordinal value that `mbstring` will use, which is the ASCII `?` character. The next call passes the hexadecimal literal `0xFFFD`, which means "use the Unicode U+FFFD character". U+FFFD is recommended by the Unicode standard; indeed, that particular character is named "Replacement Character" and intended for use as such. See https://www.unicode.org/reports/tr36/#Substituting_for_Ill_Formed_Subsequences for more detail. The UTF-8 string we pass to `mb_convert_encoding()` has an invalid two-byte sequence embedded in it, and the output contains the UTF-16 encoded substitute character (the UTF-16 encoding for U+FFFD is, of course, *0xFF 0xFD*). You might have noticed that the output string also contains the sequence *0x00*